23I501 – DESIGN AND ANALYSIS OF ALGORITHMS

BATCH 3 | DATABASE SYSTEMS: ORDERING JOINS IN A QUERY OPTIMISER

Scenario and objective. Join five relations in the fixed chain R1 ■ R2 ■ R3 ■ R4 ■ R5. **Dimensions:** p = [40, 20, 30, 10, 30, 15]. Find the parenthesisation with minimum total join cost. Joining Ri…Rk with R(k+1)…Rj costs p[i–1] × p[k] × p[j]. **Problem class:** Optimisation. **Selected technique:** Dynamic Programming (Matrix Chain Multiplication), because the problem has optimal substructure and overlapping subproblems.

1. Data structure comparison

Structure	Key operations / time	Space	Suitability
2D DP cost + split arrays	Read/update a state: O(1); retrieve chosen split: O(1).	$O(n^2)$	**Chosen.** Stores each continuous subchain's minimum cost and the split needed to reconstruct the optimal order.
Hash map (memoisation)	Average lookup/insert: O(1); hashing overhead.	$O(n^2)$ states	Can memoise subchains, but is less convenient than arrays when the DP table is dense.
Adjacency list	Neighbour traversal: O(degree).	O(V+E)	Useful for arbitrary join graphs, but unnecessary for the given fixed linear chain.

2. Design technique comparison

Technique	Assessment and evidence
Dynamic Programming	**Selected.** Optimal substructure: an optimal chain split contains optimal solutions to both smaller subchains. Overlapping subproblems are stored and reused.
Greedy	Rejected: cheapest adjacent pair first gives 30,000, while DP gives 24,500. A locally cheapest merge does not guarantee the global minimum.
Divide and Conquer	Rejected as plain recursion: different parenthesisations can recompute the same subchain costs. Memoising repeated states leads to DP.
Backtracking	Rejected for scale: exhaustive enumeration considers Catalan(n–1) orders — 14 for 5 relations and 2,674,440 for 15.
Branch and Bound	Not selected: pruning depends on useful lower bounds. Without a strong bound, many partial orders may remain; chain DP solves subchains systematically in polynomial time.

3. Result comparison on the given instance

Method	Join sequence / cost breakdown	Total cost
Dynamic Programming	R2■R3 = 6,000; R1■(R2■R3) = 8,000; R4■R5 = 4,500; final join = 6,000.	**24,500**
Greedy adjacent pair	R4■R5 = 4,500; then R3 = 4,500; then R2 = 9,000; final R1 join = 12,000.	30,000

DP saves 5,500 cost units (18.3% relative to Greedy). Optimal order: **((R1 JOIN (R2 JOIN R3)) JOIN (R4 JOIN R5))**.

4. Complexity and empirical comparison

Relations (n)	DP median time (s)	Greedy median time (s)
5	0.0000198	0.0000099
10	0.0000954	0.0000243
15	0.0002899	0.0000495
20	0.0009720	0.0001235
25	0.0018837	0.0001787

DP complexity: $O(n^3)$ time and $O(n^2)$ space. Total split evaluations are $\Sigma(l=2\ldots n)\ (n-l+1)(l-1) = n(n-1)(n+1)/6$; for five relations, this is 20. In the measured runs, Greedy is faster but produces a higher join cost. DP time rises with n, broadly consistent with cubic growth; small timings are affected by runtime and system overhead.

Conclusion

Dynamic Programming guarantees the minimum-cost parenthesisation for a chain by storing and reusing subchain solutions. Greedy is faster in these measurements but does not guarantee optimality. Exact DP for arbitrary join graphs requires an exponential subset state space.